Juniper

CLI coding assistant | Project report

Group Project 2 - 15 October 2026

1. System and design decisions

Juniper accepts a natural-language coding task in a terminal. An agent loop asks an LLM what to do, executes validated MCP tool calls, returns their results to the model, and streams its answer. It can inspect and edit project files and retrieve local or external documentation. The implementation supports Groq and Ollama; the comparison in this report uses two different Groq-hosted LLMs.

Design decision	Reason and implemented behavior
Separate providers from the agent	A shared streaming interface lets the same agent and tools run with Groq or Ollama. Model comparisons therefore reuse the execution pipeline.
Discover tools through MCP	A local filesystem server, hosted DeepWiki server, and separate custom RAG server expose tools without coupling the agent to their internals.
Validate before execution	The dispatcher checks discovered names and JSON argument schemas. Filesystem roots constrain file access. Tool confirmation is the default; evaluations use explicit auto mode.
Keep retrieval local and traceable	Chroma stores document embeddings. Returned snippets retain source, page, and chunk references. Cross-encoder reranking improves ordering within retrieved candidates.
Bound cloud requests and recovery	Adapters compact history, pace requests, and expose bounded retries. Evaluation checkpoints allow later resume. Full local history is retained; account limits remain external.

Evidence status

This report includes the completed baseline and intermediate diagnostic runs. The latest reliability fixes have regression coverage, but a complete live retest of the final policy is still pending after Groq quota exhaustion. Passing local tests is not proof of successful autonomous coding.

Report guide: model comparison p. 2; advanced RAG p. 3; reflection and evidence p. 4; original planning architecture p. 5; updated architecture and reasons for changes p. 6.

2. Two LLMs on the same coding tasks

Models: **openai/gpt-oss-20b** and **openai/gpt-oss-120b**, both through Groq. The baseline used four identical task definitions, three repetitions per model, fresh conversations, and isolated copies of the same initial files: 24 scored attempts. Only workspace paths differed in task prompts. All runs used the production agent, all three MCP servers (18 discovered tools), auto execution, up to eight iterations, and a 180-second task deadline. Baseline generation settings were SDK defaults.

A code-check pass means saved code met the evaluator's behavioral checks. An end-to-end pass additionally requires successful read/edit tools, required retrieval tools for the documentation task, normal completion, and no terminal agent error. Checks run outside the model's accessible fixture root.

Identical task	20B end-to-end	120B end-to-end
Fix mean calculation; raise ValueError for empty input	1 / 3	1 / 3
Extract unique ASCII words in first-seen order	3 / 3	0 / 3
Refactor sales totals into a shared helper	0 / 3	0 / 3
Implement UTF-8 greeting writer using retrieved docs	0 / 3	0 / 3
Total end-to-end passes	4 / 12 (33.3%)	1 / 12 (8.3%)
Total code-check passes	4 / 12	3 / 12

Insights from the baseline

20B completed the ASCII-word feature on all three attempts; 120B repeatedly requested unqualified tool names. Conversely, 120B produced two correct sales refactors, but subsequent requests exceeded the account's 8,000-token-per-minute allowance, preventing end-to-end passes. Both models fixed the mean only on their third attempt. Tool use, recovery, and quota constraints strongly affected outcomes; model size alone did not predict success.

Subsequent diagnostics and current limits

Two intermediate eight-run pilots scored **3/8** and **2/8**. The second had zero dispatched tool errors, but still failed on limits, Windows indentation, and ASCII/Unicode logic. A later free-tier batch with pacing, checkpoints, and syntax checks passed the bug-fix task with **both models**. The next slot hit the daily cap before tool use, and the runner paused: **2/3 attempted, 2/8 planned**, with five slots unattempted. This remains a partial retest, not a clean overall improvement estimate or general model ranking.

Sources: evaluations/model_comparison/results/2026-10-15/, results/2026-10-15-fixes/, and results/2026-10-15-free-tier/. Each retains its own report, traces, source files, and metadata.

3. Advanced RAG and insights

The chosen technique is **cross-encoder reranking**. The corpus contains FastMCP documentation and Python, JavaScript, and Java PDFs. Text is chunked and embedded with **all-MiniLM-L6-v2**; Chroma indexes 10,473 chunks. At query time, cosine search finds candidates, then **cross-encoder/ms-marco-MiniLM-L6-v2** jointly scores each query/passage pair. The server returns original snippets and references in the new order.

Why this technique

A semantically related passage can rank highly without answering the question. Reranking adds a query-aware relevance step while reusing the existing vectors. The MCP tool defaults to reranking; cosine mode remains available as the baseline. Cosine similarity and cross-encoder logits are distinct scores, and neither is a confidence percentage.

Recorded direct retrieval comparison

The documented production-stdio comparison used the same index and queries, top_k=5, and candidate_k=20 for reranking. These are the recorded manual observations in docs/rag-reranking.md, not a new experiment performed for this PDF.

Question	Cosine first result	Reranked first result
Register a FastMCP tool	Release note about tool-name validation	Tool-registration API documentation (previously second)
How does Python break stop a loop?	Definition, Python PDF p. 28	Loop example, Python PDF p. 29
JavaScript let versus const	Scope/reassignment discussion, p. 44	Variable-declaration comparison, p. 44

What we learned

Reranking made the first FastMCP result more directly useful and changed the top explanations for the language queries. The recorded check verified that all 30 returned snippets preserved their original text and references. However, a JavaScript passage still ranked second for the Python question, and some FastMCP matches addressed authentication rather than tools. A larger labeled query set is needed to measure retrieval quality.

The coding evaluation exposed a different limit: RAG results for the greeting task weakly supported Path.write_text, while DeepWiki supplied more relevant API details. No baseline documentation task completed, so improved grounded coding answers were not demonstrated. Reranking cannot retrieve missing documentation or rescue a relevant passage outside its candidate pool. Increasing the pool adds local scoring latency; source filters and better corpus coverage are practical next steps.

Sources: docs/rag-reranking.md; rag_server/retrieval/; baseline report's Retrieval and grounding section. The retrieval observations and coding pass rates measure different things.

4. Reflection and supporting evidence

What worked

The provider-neutral loop connected filesystem, external documentation, and local RAG tools through one execution path. Several coding attempts produced correct saved implementations, and the evaluation retained source and traces instead of relying on model claims. Direct retrieval comparisons showed useful changes in first-result relevance. Regression checks after the free-tier reliability work passed 242 tests, with strict type checking passing for 59 files.

What did not work

The baseline was unreliable: incorrect tool names, optional null arguments, malformed output, long histories, and provider quotas interrupted tasks. Windows line-ending mismatches could trigger an edit that damaged indentation. Tool success did not imply code correctness: the later 120B feature implementation mishandled Unicode when enforcing ASCII words. Retrieval sometimes returned related but insufficient evidence, and scanned PDF pages without extracted text limited corpus coverage.

Changes made and what we would improve

Implemented changes include tool aliases, optional-null cleanup, bounded recovery, context compaction, Windows-edit normalization, and the correct RAG interpreter. New free-tier controls pace requests and checkpoint batches for resume. Saved Python edits receive syntax checks through policy-controlled reads; failures feed back to the model and block normal completion until repaired. Syntax checks do not execute generated code or prove its behavior.

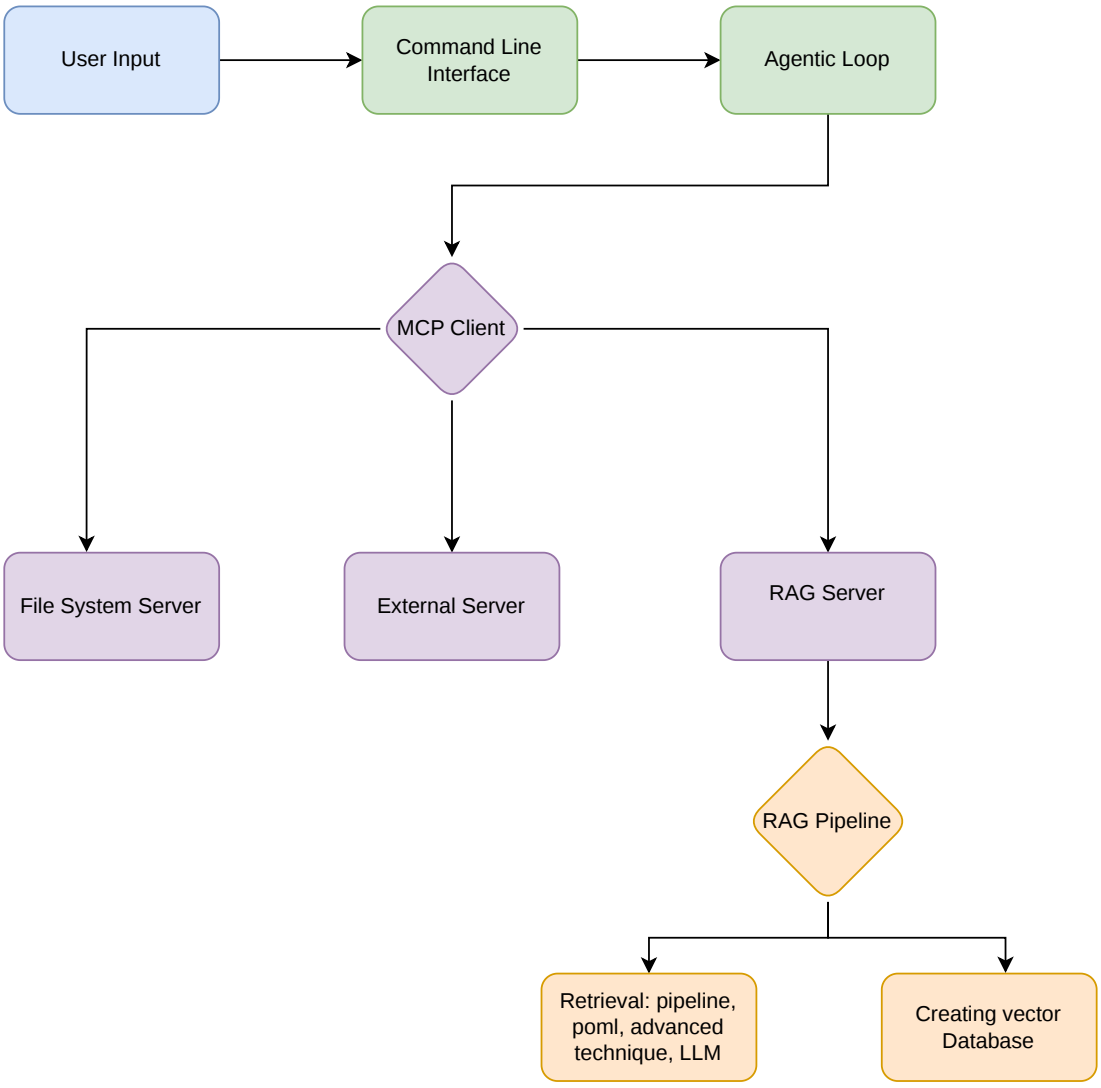
A complete current-policy live retest remains the priority, retaining failures as well as successes. We would add behavioral validation to the agent workflow, improve documentation coverage and source filtering, and evaluate reranking with independently judged queries. More repetitions, controlled account load, and recorded token usage would strengthen comparisons. These remain proposed improvements, not completed capabilities.

Evidence index (paths relative to repository root)

Evidence	Location
24-run baseline and saved code	evaluations/model_comparison/results/2026-10-15/
Fixes, 16 diagnostic runs, final-policy hashes	evaluations/model_comparison/results/2026-10-15-fixes/
RAG method and recorded observations	docs/rag-reranking.md
Free-tier controls and syntax validation	evaluations/model_comparison/FREE_TIER_VALIDATION.md
Original planning architecture (p. 5)	init-architecture.drawio2.pdf
Current structure and configuration (p. 6)	docs/architecture.md; config/mcp.json; src/coding_assistant/; rag_server/

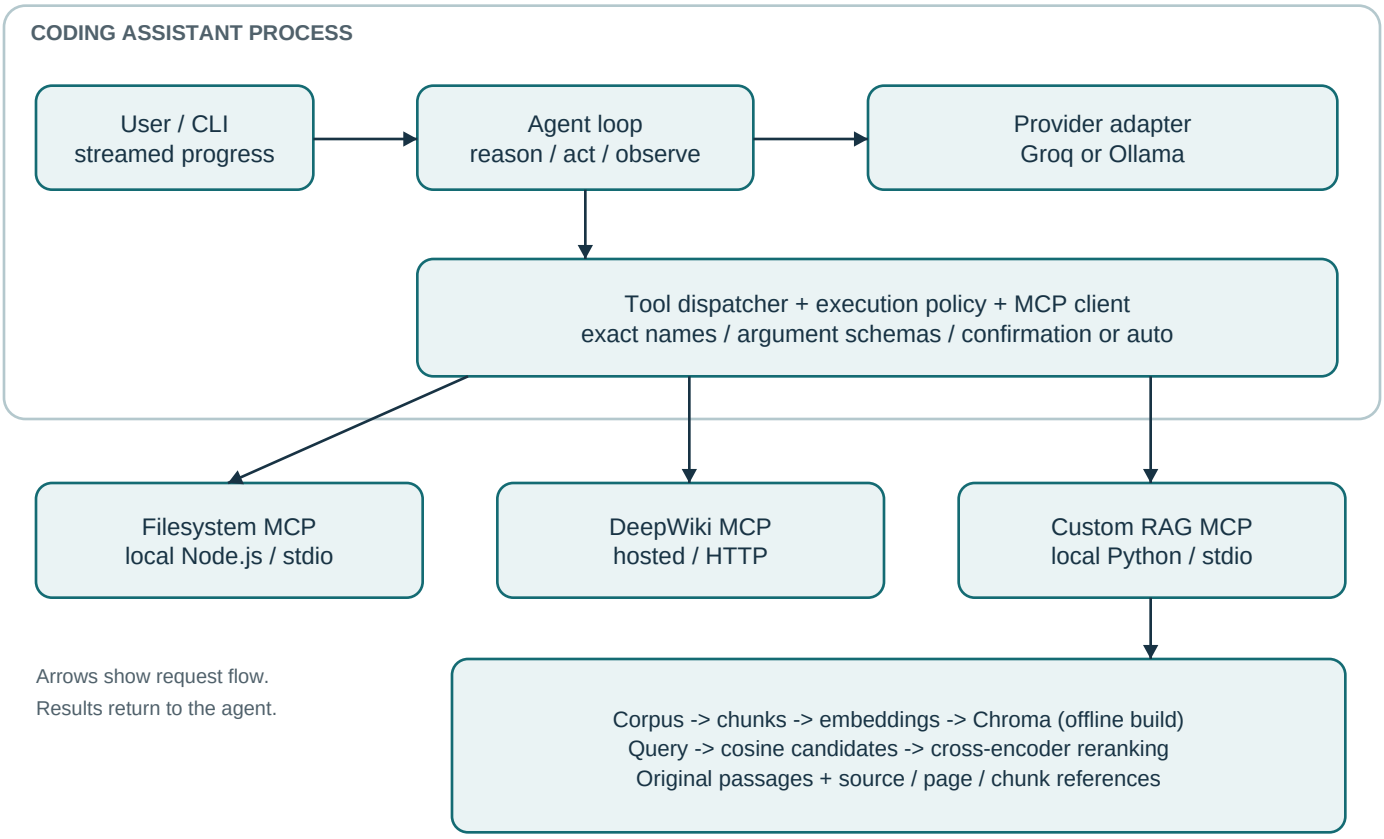
The baseline evaluated commit 6e4735d6b1b81ac2233d00d9e775ccaf5736d83b. Later fixes are local changes identified by retained source hashes. This PDF includes the three-attempt partial free-tier batch from 15 October 2026; later PowerShell runs are not incorporated automatically. Page 5 reproduces the original planning PDF as an unchanged page.

Initial Architecture



5. Updated architecture

The original planning diagram appears on page 5. The diagram below shows the implemented system, including the later reliability changes. It preserves the planned CLI, agent, MCP client, and three-server structure.



What changed from the plan, and why

Model boundary made explicit. Groq/Ollama adapters sit outside the agent's coordination logic so models can change without replacing the tool pipeline. The planning diagram did not show this boundary.

Execution checks added explicitly. Discovered tool names, schemas, and confirmation policy sit before MCP calls. Provider-side aliases, context budgeting, and retries address the failures observed during evaluation.

RAG design made concrete. The plan's generic retrieval/vector-database branches became an offline ingestion/index build and a query-time cosine-plus-cross-encoder pipeline. Answer generation remains in the main agent/provider; the RAG server returns evidence. Chroma and reranking were chosen to reuse vectors and improve passage ordering.

Deployment boundaries clarified. Filesystem and RAG are local stdio processes; DeepWiki is hosted over HTTP. The RAG executable follows the running Python interpreter to avoid missing dependencies. These refinements explain how the original architecture became executable.